

BRACT's

Vishwakarma Institute of Technology, Pune

Practical Implementation Sheet

Department: CSE-AI Semester: 5 Academic Year: 2026-27

Practical No: 2

Division: B Batch: B3

Roll No: 26 PRN No: 12413760

Name of Student: Raj Kanade

Subject Name: Deep Learning

Problem Statement:

Design and implement a Multilayer Perceptron (MLP) for classification of the Iris or Wine dataset, and evaluate its performance using accuracy and a confusion matrix.

Algorithm :

Iris Dataset

The Iris dataset is a well-known dataset used for classification tasks. It contains 150 samples of iris flowers belonging to three species:

- Iris Setosa
- Iris Versicolor
- Iris Virginica

Each sample contains four features:

1. Sepal Length (cm)
2. Sepal Width (cm)
3. Petal Length (cm)
4. Petal Width (cm)

The target variable represents the flower species.

Data Preprocessing

Before training the neural network, the dataset is preprocessed:

- **Train-Test Split:** The dataset is divided into training (80%) and testing (20%) sets.
- **Feature Scaling:** StandardScaler is used to standardise the features so that each feature has a mean of 0 and a standard deviation of 1. This improves ANN performance and convergence speed.

MLP Classifier

The model uses:

- **Hidden Layers:** (10,10)
- **Activation Function:** ReLU (Rectified Linear Unit)
- **Solver:** Adam Optimizer
- **Maximum Iterations:** 1000

ReLU Activation Function:

ReLU introduces non-linearity into the network and is defined as:

$$f(x) = \max(0, x)$$

It outputs the input directly if positive; otherwise, it outputs zero..

Adam Optimizer:

Adam (Adaptive Moment Estimation) is an optimization algorithm used to update network weights efficiently during training.

Training Process

1. Input features are fed into the network.
2. Weights and biases are initialized.
3. Forward propagation computes predictions.
4. Loss is calculated.
5. Backpropagation updates weights to minimize error.
6. The process repeats until convergence or maximum iterations are reached.

Performance Evaluation

Accuracy

Accuracy measures the percentage of correctly classified samples.

$$Accuracy = \frac{\text{Correct Predictions}}{\text{Total Predictions}}$$

Confusion Matrix

A confusion matrix shows the number of correct and incorrect predictions for each class.

Classification Report

It includes:

- **Precision:** Correct positive predictions among predicted positives.
- **Recall:** Correct positive predictions among actual positives.
- **F1-Score:** Harmonic mean of precision and recall.

Code :

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neural_network import MLPClassifier
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report
import matplotlib.pyplot as plt
import seaborn as sns
import pandas as pd

iris = load_iris()
X, y = iris.data, iris.target
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

mlp = MLPClassifier(hidden_layer_sizes=(10,10), activation='relu', solver='adam', max_iter=1000,
random_state=42)
mlp.fit(X_train, y_train)

y_pred = mlp.predict(X_test)

print("Accuracy:",accuracy_score(y_test,y_pred))
print("Confusion Matrix:\n",confusion_matrix(y_test,y_pred))
print("\nClassification Report:\n",classification_report(y_test,y_pred))

fig, axes = plt.subplots(2, 2, figsize=(16, 12))

# 1. Loss Curve
axes[0, 0].plot(mlp.loss_curve_, color='darkblue', lw=2)
axes[0, 0].set_title('MLP Training Loss Curve', fontsize=14)
axes[0, 0].set_xlabel('Epochs')
axes[0, 0].set_ylabel('Loss')
axes[0, 0].grid(True, linestyle='--', alpha=0.6)

# 2. Confusion Matrix Heatmap
cm = confusion_matrix(y_test, y_pred)
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', ax=axes[0, 1],
            xticklabels=iris.target_names, yticklabels=iris.target_names, cbar=False)
axes[0, 1].set_title('Confusion Matrix Heatmap', fontsize=14)
axes[0, 1].set_xlabel('Predicted Label')
axes[0, 1].set_ylabel('True Label')
```

3. Scatter Plot of Test Predictions (Sepal Length vs Sepal Width)

```
X_test_orig = scaler.inverse_transform(X_test)
scatter = axes[1, 0].scatter(X_test_orig[:, 0], X_test_orig[:, 1], c=y_pred, cmap='viridis', s=100, edgecolors='k')
axes[1, 0].set_title('Predicted Classes (Sepal Length vs Width)', fontsize=14)
axes[1, 0].set_xlabel(iris.feature_names[0])
axes[1, 0].set_ylabel(iris.feature_names[1])
handles, _ = scatter.legend_elements()
axes[1, 0].legend(handles, iris.target_names, title="Predicted Species")
```

4. Classification Report Metrics Bar Chart

```
report = classification_report(y_test, y_pred, target_names=iris.target_names, output_dict=True)
df_report = pd.DataFrame(report).iloc[:-1, :3].T
df_report.plot(kind='bar', ax=axes[1, 1], colormap='Set2')
axes[1, 1].set_title('Classification Report Metrics', fontsize=14)
axes[1, 1].set_ylabel('Score')
axes[1, 1].set_xticklabels(iris.target_names, rotation=0)
axes[1, 1].set_ylim(0, 1.1)
axes[1, 1].grid(axis='y', linestyle='--', alpha=0.6)

plt.tight_layout()
plt.show()
```

Output :

Evaluation matrices:

Accuracy: 0.9666666666666667

Confusion Matrix:

```
[[10  0  0]
 [ 0  8  1]
 [ 0  0 11]]
```

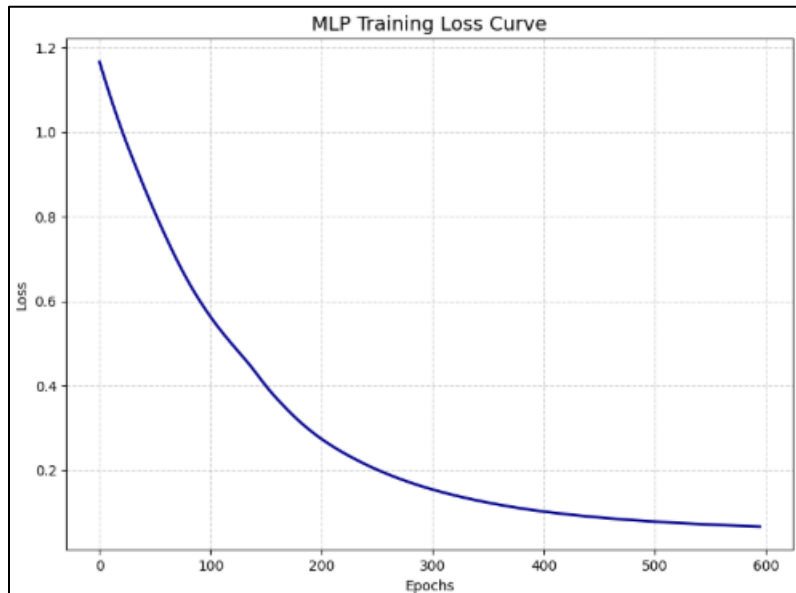
Classification Report:

	precision	recall	f1-score	support
0	1.00	1.00	1.00	10
1	1.00	0.89	0.94	9
2	0.92	1.00	0.96	11
accuracy			0.97	30
macro avg	0.97	0.96	0.97	30
weighted avg	0.97	0.97	0.97	30

Visualisation of data:

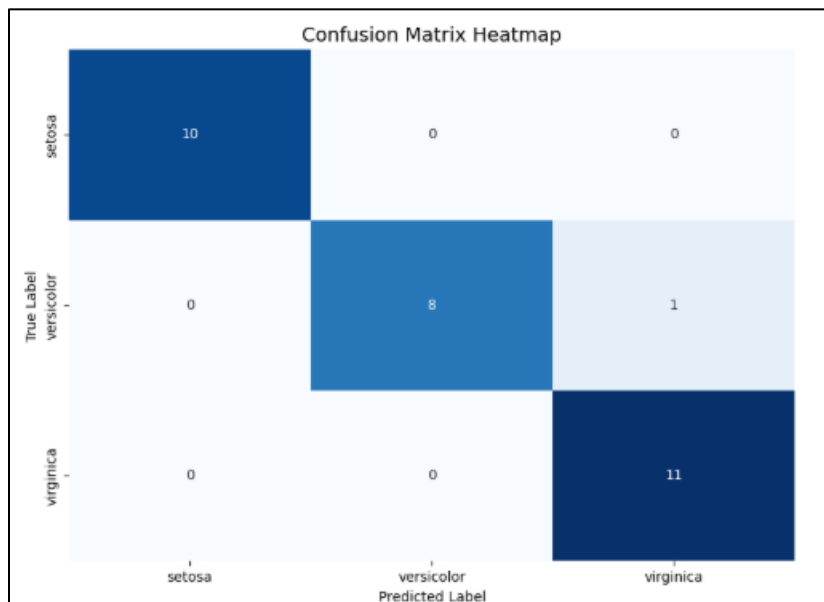
1. Loss Curve

- Shows how the loss decreases during training.
- Indicates learning progress of the ANN.



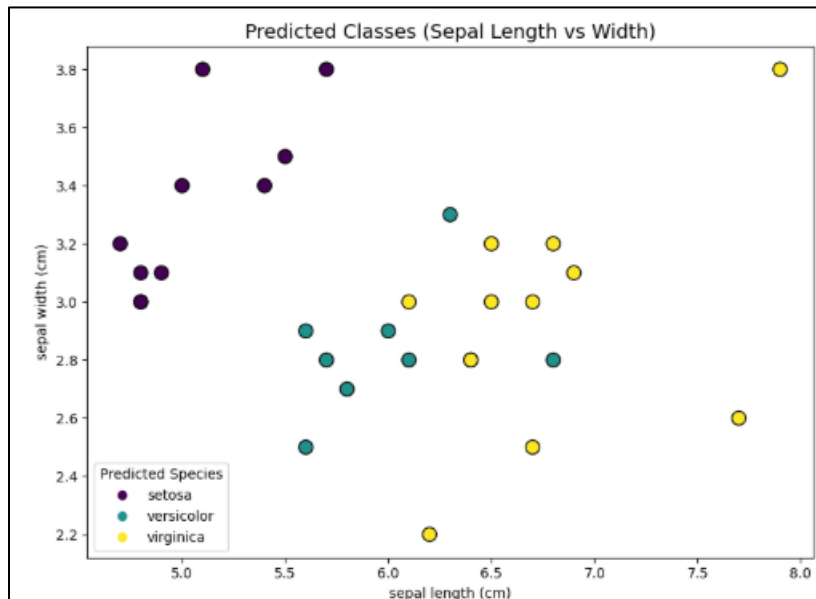
2. Confusion Matrix Heatmap

- Visual representation of classification performance.



3. Scatter Plot of Test Predictions (Sepal Length vs Sepal Width)

- Displays predicted flower species using Sepal Length and Sepal Width.



4. Classification Report Metrics Bar Chart

- Compares Precision, Recall, and F1-Score for each species.

